Chapitre : Notions préalables à la programmation asynchrone

Synchrone/asynchrone, fonctions comme valeurs, callbacks, génériques, *null safety*, `Iterable`, packages et HTTP

Pr. Abdelfattah Abassi

Département Physique Appliquée – Laboratoire LAVETE
Faculté des Sciences et Techniques de Settat
Université Hassan I

Année universitaire 2026–2027

Objectifs et organisation du chapitre

À la fin de ce chapitre, vous saurez

- distinguer exécution **synchrone** et **asynchrone**, **bloquant** et **non bloquant** ;
- écrire des **fonctions anonymes**, des fonctions **flèche** et des **callbacks** ;
- comprendre les **événements** et l'**abonnement** ;
- lire les types **génériques** (`Future<T>`, `List<T>`) ;
- manipuler `null`, `?`, `??`, `!`, `as`, `is` ;
- distinguer `Iterable` et `Stream` grâce aux générateurs ;
- importer des bibliothèques et des **packages** ;
- comprendre client/serveur, URL et JSON comme texte.

Place dans le cours

À enseigner **avant** le chapitre « Programmation asynchrone, exceptions et API REST ».

Prérequis

Variables, types de base, conditions, boucles, fonctions simples, classes simples, listes et dictionnaires (chapitre « Le langage Dart »).

Méthode

idée → *exemple* → *exercice*.

Plan du chapitre

1. Synchrone et asynchrone
2. Fonctions comme valeurs
3. Callbacks et événements
4. Types génériques
5. *Null safety* et types dynamiques
6. Collections, `Iterable` et générateurs
7. Classes pour modéliser les données
8. Bibliothèques et packages
9. Réseau et HTTP : les bases
10. Synthèse et auto-évaluation

Pourquoi des types génériques ?

```dart
class Boite<T> {
  final T contenu;
  Boite(this.contenu);

  T ouvrir() => contenu;
}

void main() {
  var a = Boite<int>(42);
  var b = Boite<String>('Sara');

  print(a.ouvrir() + 1);        // 43
  print(b.ouvrir().length);     // 4
}
```

T : un type à choisir

`T` est un **paramètre de type** : une même classe fonctionne pour **tous** les types, tout en restant **typée**.

Intérêt

`a.ouvrir()` est un `int` : le compilateur **vérifie** nos usages.

Lire les types génériques

Type	Lecture
`List<int>`	liste d'entiers
`Map<String, int>`	clés texte, valeurs entières
`Map<String, dynamic>`	clés texte, valeurs variées
`Future<int>`	un entier **plus tard**
`Future<void>`	action terminée plus tard
`Stream<double>`	réels **au fil du temps**
`Future<List<Todo>>`	liste de `Todo` plus tard

```dart
T premier<T>(List<T> l) => l.first;

void main() {
  print(premier<int>([4, 5]));    // 4
  print(premier(['a', 'b']));     // a
}
```

Fonction générique

Le type `T` est souvent **déduit** automatiquement.

Exercices : génériques

1. Écrivez une classe `Paire<A, B>` avec deux attributs, `premier` de type `A` et `second` de type `B`. Créez une `Paire<String, int>`.
2. Écrivez la fonction générique `T dernier<T>(List<T> l)`.
3. Traduisez en français : `Future<bool>`, `Map<String, List<int>>`, `Stream<String>`.
4. Pourquoi `List<int> l = [1, 2]; l.add('a');` est-il refusé par le compilateur ?

Plan du chapitre

1. Synchrone et asynchrone

2. Fonctions comme valeurs

3. Callbacks et événements

4. Types génériques

5. *Null safety* et types dynamiques

6. Collections, `Iterable` et générateurs

7. Classes pour modéliser les données

8. Bibliothèques et packages

9. Réseau et HTTP : les bases

10. Synthèse et auto-évaluation

null et types nullables

```dart
void main() {
  String nom = 'Sara';
  // String autre = null;     // ERREUR : String refuse null

  String? surnom;             // String? : texte OU null
  print(surnom);              // null

  surnom = 'Sasa';
  print(surnom);              // Sasa
}
```

Sortie :

```
null
Sasa
```

Null safety

Par défaut, un type **n'accepte pas** `null`. Le point d'interrogation ? l'**autorise**.

Pourquoi ?

Éviter le plantage classique : appeler une méthode sur une valeur **absente**.

Opérateurs ?., ??, ??= et !

```dart
Map<String, String> m = {'nom': 'Sara'};
String? ville = m['ville'];        // clé absente -> null

print(ville?.length);              // null (pas d'erreur)
print(ville ?? 'inconnue');        // inconnue
ville ??= 'Settat';                // affecte si null
print(ville);                      // Settat

int n = int.tryParse('abc') ?? 0; // 0
String t = m['nom']!;              // ! : je garantis non null
print('$n $t');                    // 0 Sara
```

`T?`	peut être **null**
`a?.b`	accès sûr
`a ?? b`	valeur par défaut
`a ??= b`	affecte si **null**
`a!`	affirme non **null**

Danger de !

Si la valeur est **null**, le programme **plante**. À utiliser seulement si on en est **sûr**.

Après `if (x != null)`, `x` est traité comme **non nul**.

dynamic, is, as et num

```
dynamic valeur = 21;              // type connu à l'exécution
print(valeur is int);             // true
valeur = 'texte';                 // permis avec dynamic

dynamic d = 20;
int n = d as int;                 // cast : TypeError si faux

num mesure = 23;                  // num : int OU double
mesure = 23.5;
double x = mesure.toDouble();     // 23.5
print('$n $x');                   // 20 23.5
```

Trois outils

`dynamic` : type **non vérifié** avant l'exécution.
`is` : **teste** le type.
`as` : **convertit** (cast) le type.

num

Type **parent** de `int` et `double`.
Utile quand une mesure peut être 23 ou 23.5.

Application : lire une Map hétérogène

```dart
Map<String, dynamic> data = {
  'nom': 'Sara',
  'age': 20,
  'notes': [15, 17.5],
};

String nom = data['nom'] as String;
int age = data['age'] as int;
List<dynamic> notes = data['notes'];
double n = (notes[1] as num).toDouble();

print('$nom, $age ans, note : $n');
```

Sortie :

```
Sara, 20 ans, note : 17.5
```

Le même schéma que JSON

Un JSON décodé donne une `Map<String, dynamic>` : on **convertit** chaque valeur avec `as`.

Attention

`data['notes'][1] as int` échouerait : `17.5` n'est pas un `int`. D'où `num`.

Exercices : *null safety* et types

1. Déclarez `String? ville` sans valeur ; affichez sa longueur de façon **sûre**, puis `'inconnue'` si elle est absente.
2. Convertissez `'12'`, `'abc'` et `'3.5'` en entiers avec `int.tryParse`, avec `0` par défaut.
3. Soit `Map<String, dynamic> e = {'nom': 'Omar', 'age': 22, 'moyenne': 14}` : extrayez chaque valeur avec le bon type, la moyenne en `double`.
4. Que se passe-t-il avec `dynamic d = 'abc'; int n = d as int;` ?
5. Écrivez `String decrire(Object o)` qui renvoie « entier », « texte » ou « autre » avec `is`.

Plan du chapitre

1. Synchrone et asynchrone
2. Fonctions comme valeurs
3. Callbacks et événements
4. Types génériques
5. *Null safety* et types dynamiques
6. Collections, `Iterable` et générateurs
7. Classes pour modéliser les données
8. Bibliothèques et packages
9. Réseau et HTTP : les bases
10. Synthèse et auto-évaluation

Rappel : List, Map et boucles for

```dart
List<int> notes = [12, 8, 15];
notes.add(9);
print(notes[0]);              // 12
print(notes.length);          // 4

Map<String, int> ages = {'Sara': 20, 'Omar': 22};
ages['Lina'] = 19;
print(ages['Omar']);          // 22  (de type int?)

for (final n in notes) {
  print(n);
}
for (final e in ages.entries) {
  print('${e.key} : ${e.value}');
}
```

À retenir

`List` : éléments **ordonnés**, accès par **indice**.
`Map` : paires **clé** → **valeur**, accès par **clé**.
`for (final x in c)` parcourt une collection.

Lien avec *null safety*

`ages['Omar']` est de type `int?` : la clé peut être absente.

Méthodes fonctionnelles sur les collections

```
var notes = [12, 8, 15, 6, 17];

var bonnes = notes.where((n) => n >= 10).toList();
print(bonnes);                    // [12, 15, 17]

var doubles = notes.map((n) => n * 2).toList();
print(doubles);                   // [24, 16, 30, 12, 34]

int somme = notes.reduce((a, b) => a + b);
print(somme);                     // 58

print(notes.take(2).toList());    // [12, 8]
notes.forEach((n) => print(n));
```

`where`	filtre
`map`	transforme
`reduce`	combine
`take` / `skip`	garde / saute *n*
`toList`	→ `List`
`forEach`	agit sur chacun

Piège

`map` et `where` renvoient un `Iterable`, **pas** une `List` : ajouter `.toList()`.

Ces mêmes noms existent pour les `Stream`.

Générateur synchrone : `sync*` et `yield`

Principe

Un générateur est une fonction qui produit plusieurs valeurs progressivement, au lieu de retourner toutes les valeurs en une seule fois.

```
Iterable<int> compter(int max) sync* {
    for (int i = 1; i <= max; i++) {
        print('Production de $i');
        yield i;
    }
}

void main() {
    for (final v in compter(3)) {
        print('Recu $v');
    }
}
```

Comprendre la déclaration du générateur

```
Iterable<int> compter(int max) sync*
```

`Iterable<int>` Séquence de valeurs entières que l'on peut parcourir.

`compter` Nom de la fonction génératrice.

`int max` Paramètre indiquant la limite du comptage.

`sync*` Indique que la fonction est un générateur synchrone.

Exemple

`compter(3)` permet de parcourir les valeurs :

1 2 3

Le rôle de `yield`

```
for (int i = 1; i <= max; i++) {
    print('Production de $i');
    yield i;
}
```

Que fait `yield i` ?

1. Produit la valeur actuelle.
2. Transmet cette valeur à celui qui parcourt le générateur.
3. Suspend temporairement l'exécution de la fonction.
4. Reprend l'exécution lorsque la prochaine valeur est demandée.

Attention

`yield` ne termine pas la fonction : il la suspend temporairement. La fonction conserve son état

Comment fonctionne la boucle `for` ?

```
void main() {
    for (final v in compter(3)) {
        print('Recu $v');
    }
}
```

1. `compter(3)` fournit un générateur.
2. La boucle `for` demande la première valeur.
3. Le générateur exécute son code jusqu'au premier `yield`.
4. La valeur produite est affectée à `v`.
5. La boucle affiche cette valeur.
6. La boucle demande ensuite la valeur suivante.

Idée essentielle

Le générateur ne produit pas les trois valeurs simultanément : il les produit une par une, à la demande de l'itérateur.

Simulation de l'exécution

Générateur

```
yield 1;
yield 2;
yield 3;
```

Boucle principale

```
for (final v in compter(3)) {
    print('Recu $v');
}
```

Déroulement

1. Produit 1 → Reçoit 1
2. Produit 2 → Reçoit 2
3. Produit 3 → Reçoit 3
4. Fin du générateur.

Ordre d'exécution

La production et la réception alternent : **Production 1, Réception 1, Production 2, Réception 2**

Résultat de l'exécution

```
Iterable<int> compter(int max) sync* {
    for (int i = 1; i <= max; i++) {
        print('Production de $i');
        yield i;
    }
}

void main() {
    for (final v in compter(3)) {
        print('Recu $v');
    }
}
```

Sortie :

```
Production de 1
Recu 1
Production de 2
Recu 2
Production de 3
Recu 3
```

Pourquoi cet ordre ?

Après chaque `yield`, le générateur est suspendu. La boucle reçoit la valeur, l'affiche, puis demande la suivante.

Différence entre `return` et `yield`

Avec `return`

- Retourne un résultat.
- Termine la fonction.
- Ne permet pas de reprendre après le retour.

```
int obtenir() {
    return 10;
}
```

Avec `yield`

- Émet une valeur.
- Suspend la fonction.
- Permet de reprendre et d'émettre d'autres valeurs.

```
Iterable<int> obtenir() sync* {
    yield 10;
    yield 20;
    yield 30;
}
```

À retenir

`return` termine la fonction, tandis que `yield` permet de produire plusieurs valeurs successivement.

Générateurs synchrones et asynchrones

Dart propose deux types de générateurs :

Caractéristique	`sync*`	`async*`
Type retourné	Iterable	Stream
Fonctionnement	Synchrone	Asynchrone
Émission	yield	yield
Parcours	for-in	await for

Quand utiliser ?

- `sync*` : produire une séquence de valeurs synchrones.
- `async*` : produire des événements au fil du temps, notamment lors d'opérations asynchrones.

Exercice : comprendre l'ordre d'exécution

```
Iterable<int> nombres() sync* {
    yield 10;
    print("A");
    yield 20;
    print("B");
    yield 30;
}

void main() {
    for (var n in nombres()) {
        print(n);
    }
}
```

Question

Maintenant ou plus tard ? Le pont vers `Future` et `Stream`

	Une valeur	Plusieurs valeurs
Maintenant (synchrone)	`T`	`Iterable<T>`, `List<T>`
Plus tard (asynchrone)	`Future<T>`	`Stream<T>`

Générateurs et boucles

`sync*` → `Iterable` : parcouru par `for`.

`async*` → `Stream` : parcouru par `await for`.

À retenir

`yield` fonctionne de la même façon ; la seule différence : les valeurs peuvent **arriver avec du retard**, d'où `await`.

Exercices : collections et générateurs

1. À partir de `[18, 25, 31, 22, 35]`, gardez les températures supérieures à 24 et calculez leur moyenne.
2. Écrivez `Iterable<int> pairs(int max) sync*` qui produit les nombres pairs de 2 à `max`.
3. Ajoutez un `print` dans le générateur et expliquez l'ordre d'affichage (évaluation à la demande).
4. Dites pourquoi `[1, 2, 3].map((x) => x * 2)` n'est pas une liste et corrigez.
5. Complétez : `String` est à `Future<String>` ce que `List<String>` est à ...

Plan du chapitre

1. Synchrone et asynchrone

2. Fonctions comme valeurs

3. Callbacks et événements

4. Types génériques

5. *Null safety* et types dynamiques

6. Collections, `Iterable` et générateurs

7. Classes pour modéliser les données

8. Bibliothèques et packages

9. Réseau et HTTP : les bases

10. Synthèse et auto-évaluation

final, required et toString

```dart
class Etudiant {
  final String nom;
  final int age;

  Etudiant({required this.nom, required this.age});

  @override
  String toString() => '$nom ($age ans)';
}

void main() {
  var e = Etudiant(nom: 'Sara', age: 20);
  print(e);
}
```

Sortie :

```
Sara (20 ans)
```

Vocabulaire

`final` : non modifiable après construction.
`this.nom` : raccourci d'affectation.
`required` : paramètre nommé obligatoire.

@override

`toString` définit le **texte affiché** par `print`.

Constructeur nommé et `factory`

```dart
class Point {
  final double x, y;

  Point(this.x, this.y);

  // constructeur nommé
  Point.origine() : x = 0, y = 0;

  // constructeur factory : calcule puis construit
  factory Point.depuisMap(Map<String, dynamic> m) =>
      Point((m['x'] as num).toDouble(),
            (m['y'] as num).toDouble());
}

void main() {
  var o = Point.origine();
  var p = Point.depuisMap({'x': 1, 'y': 2.5});
  print('${o.x} / ${p.x}, ${p.y}');   // 0.0 / 1.0, 2.5
}
```

Constructeur nommé

Plusieurs **façons** de créer un objet : `Point.origine()`.

`factory`

Constructeur qui peut **calculer ou convertir** avant de construire. C'est le modèle de `Todo.fromJson(...)`.

abstract, implements et classes d'erreur

```
abstract class Capteur {
  double lire();                  // contrat, sans corps
}
class CapteurTemperature implements Capteur {
  @override
  double lire() => 23.5;
}
class PanneException implements Exception {
  final String message;
  PanneException(this.message);
  @override
  String toString() => 'Panne : $message';
}
void main() {
  Capteur c = CapteurTemperature();
  print(c.lire());                          // 23.5
  print(PanneException('hors service'));  // Panne : hors service
}
```

Contrat

`abstract class` décrit **ce que** doit offrir une classe. `implements` **s'engage** à le fournir.

Exceptions personnalisées

`Exception` est aussi un contrat : `implements Exception` crée un type d'erreur dédié.

Exercices : classes

1. Créez une classe `Mesure` avec `valeur` (`double`) et `unite` (`String`), paramètres nommés requis, et un `toString` lisible.
2. Ajoutez un constructeur `factory Mesure.depuisMap(Map<String, dynamic> m)` et testez-le avec `{'valeur': 23, 'unite': 'C'}`.
3. Ajoutez une méthode `Map<String, dynamic> toMap()` qui fait l'opération inverse.
4. Définissez `abstract class Forme` avec `double aire()` puis deux classes qui l'implémentent.
5. Créez `AgeInvalideException implements Exception` avec un message.

Plan du chapitre

1. Synchrone et asynchrone
2. Fonctions comme valeurs
3. Callbacks et événements
4. Types génériques
5. *Null safety* et types dynamiques
6. Collections, `Iterable` et générateurs
7. Classes pour modéliser les données
8. Bibliothèques et packages
9. Réseau et HTTP : les bases
10. Synthèse et auto-évaluation

Les bibliothèques standard de Dart

Bibliothèque	Import	Contenu
`dart:core`	automatique	`print`, `String`, `List`, `Duration`, `Future`, `Stream`
`dart:async`	`import`	`Timer`, `StreamController`, `TimeoutException`
`dart:convert`	`import`	`jsonDecode`, `jsonEncode`
`dart:io`	`import` (pas sur le web)	`sleep`, `SocketException`, fichiers
`dart:math`	`import`	`Random`, `sqrt`, `pow`
`package:http`	à **installer**	`http.get`, `http.post`

Remarque

Les types `Future` et `Stream` sont disponibles **sans import**. Les outils plus avancés (`Timer`, `StreamController`, `TimeoutException`) nécessitent `import 'dart:async';`.

Importer et installer un package

```dart
import 'dart:math';                          // tout dart:math
import 'dart:convert' show jsonDecode;       // seulement jsonDeco
import 'package:http/http.dart' as http;     // préfixe http

void main() {
  print(Random().nextInt(10));               // un entier de 0 à 9
  // http.get(...) : on utilise le préfixe
}
```

Dans pubspec.yaml

```yaml
dependencies:
  http: ^1.2.0
```

Installer

En terminal : `dart pub add http` (ou `flutter pub add http`).
Les packages se cherchent sur `pub.dev`.

Options d'import

`as p` : **préfixe**, évite les conflits de noms.

`show x` : n'importe **que** `x`.

`hide x` : importe tout **sauf** `x`.

Exercices : bibliothèques

1. Affichez 5 nombres aléatoires entre 0 et 99 avec `dart:math`.
2. Dans quelle bibliothèque trouve-t-on `jsonDecode` ? `Timer` ? `sleep` ?
3. Ajoutez le package `http` à un projet et vérifiez `pubspec.yaml`.
4. Pourquoi écrit-on `http.get(...)` et non `get(...)` ?

Plan du chapitre

1. Synchrone et asynchrone

2. Fonctions comme valeurs

3. Callbacks et événements

4. Types génériques

5. *Null safety* et types dynamiques

6. Collections, `Iterable` et générateurs

7. Classes pour modéliser les données

8. Bibliothèques et packages

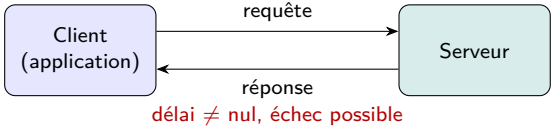
9. Réseau et HTTP : les bases

10. Synthèse et auto-évaluation

Client, serveur et latence

Client / serveur

Le **client** (votre application) envoie une **requête** ; le **serveur** traite et renvoie une **réponse**.



Latence

Le trajet prend du **temps** (réseau, traitement) et peut **échouer** : c'est exactement le cas d'usage de l'**asynchrone**.

Anatomie d'une URL

`https://jsonplaceholder.typicode.com/todos?_limit=5`

Partie	Exemple
protocole	`https` (`http` sécurisé)
hôte (serveur)	`jsonplaceholder.typicode.com`
chemin (ressource)	`/todos`
paramètres	`?_limit=5` (*clé*=*valeur*)

À essayer

Ouvrez cette adresse dans le **navigateur** : vous voyez directement la **réponse** du serveur, au format JSON.

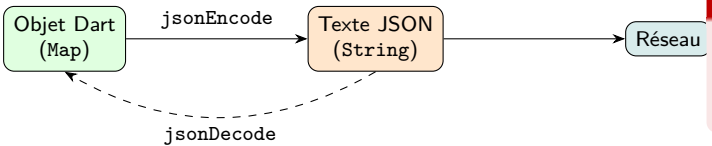
Côté Dart

`Uri.parse('...')` transforme le texte en objet `Uri`.

Les données circulent sous forme de texte

```dart
String texte = '{"id": 1, "titre": "Réviser"}';

print(texte.runtimeType);      // String : un TEXTE
// print(texte['id']);         // erreur : ce n'est pas une Map
```



Sérialisation

Sérialiser : objet → texte.
Désérialiser : texte → objet.

À retenir

La réponse d'un serveur arrive d'abord comme un `String` ; il faut la **décoder** avant de l'utiliser.

Exercices : réseau et texte

1. Ouvrez dans le navigateur `https://jsonplaceholder.typicode.com/users/1` et listez les clés du JSON affiché.
2. Décomposez l'URL `https://api.exemple.com/mesures?capteur=12&limite=10` (protocole, hôte, chemin, paramètres).
3. Écrivez à la main le JSON d'un étudiant (nom, âge, liste de notes).
4. Quel est le type de `'{"a": 1}'` en Dart ? Comment obtenir la valeur de `a` ?
5. Citez trois causes possibles d'échec d'une requête réseau.

Plan du chapitre

1. Synchrone et asynchrone
2. Fonctions comme valeurs
3. Callbacks et événements
4. Types génériques
5. *Null safety* et types dynamiques
6. Collections, `Iterable` et générateurs
7. Classes pour modéliser les données
8. Bibliothèques et packages
9. Réseau et HTTP : les bases
10. Synthèse et auto-évaluation

Auto-évaluation (1/2) : êtes-vous prêt ?

1. Que provoque `sleep(Duration(seconds: 3))` dans une application mobile ?
2. Écrivez en syntaxe flèche une fonction anonyme qui renvoie le triple de `x`.
3. Quel est le type de `(String s) => s.length` ?
4. Que vaut `ville ?? 'inconnue'` si `ville` est `null` ? Si elle vaut `'Rabat'` ?
5. `[1, 2, 3].map((x) => x * 2)` renvoie-t-il une `List` ?

Auto-évaluation (2/2)

6. Comment lit-on `Future<List<String>>` ?
7. Quelle différence entre `String` et `String?` ?
8. Que fait `valeur as int` et quand échoue-t-il ?
9. Que signifie `import 'package:http/http.dart' as http;` ?
10. Quelle différence entre `Iterable<int>` et `Stream<int>` ?

Corrigé de l'auto-évaluation

1. Il **bloque** le programme 3 s : l'interface est figée.
2. `var triple = (int x) => x * 3;`
3. `int Function(String)`
4. `'inconnue'` puis `'Rabat'`.
5. Non : un `Iterable` ; ajouter `.toList()`.
6. Une **liste de textes**, disponible **plus tard**.
7. `String` refuse `null` ; `String`? l'accepte.
8. Il **convertit** la valeur en `int` ; échec (`TypeError`) si ce n'est pas un entier.
9. Importe le package `http` ; ses fonctions s'utilisent avec le préfixe `http.`
10. `Iterable` : valeurs disponibles **maintenant** ; `Stream` : valeurs qui arrivent **au fil du temps**.

Récapitulatif du chapitre

Temps et fonctions

Synchrone / asynchrone, bloquant, latence ; fonctions anonymes, flèche, paramètres nommés ; callbacks et événements.

Collections et classes

`Iterable`, `sync*` / `yield` ; `final`, `required`, `factory`, `abstract`, `implements`.

Types

Génériques `<T>` ; `T?`, `?.`, `??`, `!` ; `dynamic`, `is`, `as`, `num`.

Environnement

Bibliothèques `dart:`, packages et `pub` ; client/serveur, URL, JSON comme texte.

Pont vers le chapitre suivant

Suite

Chapitre « Programmation asynchrone, exceptions et API REST » : `Future`, `async`/`await`, `Stream`, `try`/`catch`, JSON et `http`.

Condition

Passez au chapitre suivant quand l'auto-évaluation est réussie **sans** consulter le corrigé.